

IT5103N Advanced Mobile Development (Final Project)

Objective:

Develop a React Native mobile app called "PokeExplorer" that serves as an interactive, augmented reality-enhanced Pokedex. The app allows users to explore, "discover," and catalog Pokémon in real-world contexts using geolocation, AR overlays, and device sensors, while promoting community sharing and educational insights about Pokémon species. The app must be built to run natively on both iOS and Android platforms.

Purpose:

The app aims to blend education, entertainment, and real-world interaction by turning Pokémon discovery into a gamified learning experience. Users can learn about Pokémon attributes (e.g., types, abilities, habitats) from PokeAPI while engaging with their surroundings through AR/VR elements, geolocation-based "hunts," and multimedia features. This fosters skills in mobile development, API integration, and hardware utilization, while encouraging social collaboration (e.g., sharing discoveries). It's designed to be lightweight and educational, avoiding complex 3D graphics like Pokémon Go, focusing instead on simple overlays, maps, and media to make it achievable in 1 month for student groups balancing other coursework. The purpose ties into environmental awareness by mapping Pokémon to real-world locations (e.g., water-type near bodies of water), promoting outdoor exploration and digital literacy.

Key Information:

Duration: 6 weeks from today.

Demo day: (I'll announce later)

Late submission = -20 points/day.

Requirements

Use bare React Native for cross-platform development (iOS/Android), ensuring compatibility through platform-specific code where needed (e.g., using Platform.OS checks). Leverage free libraries for hardware access. Integrate PokeAPI for Pokémon data. Total estimated effort: 4 weeks for core MVP, 1~2 weeks for polish and extras. Setup bare React Native project using react-native init, and handle iOS/Android builds with Xcode and Android Studio. Use github and make sure all members of the group commit. Consider approval and merging policy.

Functional Requirements

1. User Authentication and Profiles (Basic Setup)

- Simple login/signup using email or social providers (e.g., via React Native Firebase).
- User profile to track discovered Pokémon, with a personal Pokedex list.

2. Pokedex Core (API Integration)

- Fetch and display Pokémon data from PokeAPI (e.g., name, types, abilities, stats, sprites).
- Search functionality: Allow users to search by name, type, or ID.
- Detail view: Show full info, including evolution chains (if time) and flavor text.
- Offline support: Cache recent searches using AsyncStorage or Realm for basic data persistence.

3. Geolocation-Based Discovery

- Use device GPS (via @react-native-community/geolocation or react-native-geolocation-service) to detect user location.
- "Hunt" mode: Simulate Pokémon encounters based on location (e.g., random Pokémon appear on a map view using react-native-maps; tie to real-world biomes like urban/rural via simple logic or Google Maps API if needed).
- Notification alerts for nearby "Pokémon" (using react-native-push-notification).

4. AR/VR Elements (Keep Simple)

- AR integration: Use libraries like ViroReact to overlay Pokémon sprites/images on the camera feed (e.g., point camera at a landmark to "spawn" a Pokémon).
- VR-lite: Implement a 360-degree view of Pokémon habitats using panoramic images or simple gyroscopic controls (via react-native-sensors for device orientation).
- Avoid complex 3D models; use 2D sprites/GIFs from PokeAPI or free assets.

5. Camera and Mic Integration

- Camera: Allow users to "capture" Pokémon by taking photos with AR overlays (e.g., superimpose Pokémon image on real-world photo using react-native-vision-camera).
- Mic: Voice search for Pokémon (e.g., speak a name to query PokeAPI via react-native-voice).

- Save captures to a gallery within the app.

6. Multimedia Loading

- Load and display GIFs/images: Fetch Pokémon sprites/GIFs from PokeAPI or external sources (e.g., Giphy for animations).
- Lazy loading for performance (using React Native's Image component with caching).

7. Sharing and Social Features

- Share discoveries: Export Pokémon details, captured photos, or stats to social media (e.g., Twitter, Instagram) or messaging apps (WhatsApp, etc.) via react-native-share.
- Community feed: Basic in-app feed where users can post and view shared discoveries (using Firebase Realtime Database for simplicity).

8. Gamification and Progression (Option/Bonus)

- Badges or points for discoveries.
- Daily challenges tied to geolocation (e.g., "Find a fire-type today").

Non-Functional Requirements

- Performance: App should run smoothly on mid-range devices; optimize API calls (e.g., batch requests) and image loading.
- UI/UX: Clean, intuitive design using React Native components (e.g., FlatList for Pokedex). Theme inspired by Pokémon (simple colors, no heavy animations).
- Security: Handle user data securely (e.g., no plain-text storage); API keys hidden in .env.
- Testing: Unit tests for API integrations (Jest); manual testing for hardware features. Ensure cross-platform compatibility with iOS and Android, including handling permissions (e.g., location, camera) via react-native-permissions.
- Documentation: Groups must submit code repo (GitHub), a demo video, and a report on challenges (e.g., AR integration issues).
- Scalability and Constraints: Limit to 100-200 Pokémon for initial load; no backend server required beyond Firebase for sharing.

- Tools/Libraries: React Native (latest stable), PokeAPI, React Navigation for routing, Redux or Context for state management, and the specific libraries mentioned above for hardware features. Ensure all libraries are compatible with bare React Native workflows.

Grading Rubric for PokeExplorer Mobile App

1. Functionality (40 points)

Evaluates the implementation of the required features and their correctness.

- User Authentication and Profiles (8 points)
 - 8: Fully functional login/signup with email or social providers (e.g., Firebase); user profile tracks discovered Pokémon accurately.
 - 6: Login/signup works with minor bugs; profile partially functional (e.g., tracks some data).
 - 4: Basic authentication implemented but unstable or incomplete profile.
 - 2: Authentication attempted but non-functional or profile missing.
 - 0: No authentication or profile implemented.
- Pokedex Core with API Integration (10 points)
 - 10: PokeAPI fully integrated; displays name, types, abilities, stats, sprites; search by name/type/ID; offline caching works.
 - 8: PokeAPI integrated with minor issues; search functional but limited (e.g., no caching or partial data display).
 - 6: Basic PokeAPI integration; displays limited data; search buggy or incomplete.
 - 4: API integration attempted but inconsistent; minimal data shown.
 - 0: No API integration or Pokedex functionality.
- Geolocation-Based Discovery (10 points)
 - 10: Geolocation (via react-native-geolocation-service or similar) detects location; map view (react-native-maps) shows Pokémon tied to biomes; notifications work.
 - 8: Geolocation and map work with minor bugs; biome logic partial; notifications functional.
 - 6: Geolocation works but map or notifications have significant issues; biome logic missing.

- 4: Geolocation attempted but unreliable; no map or notifications.
 - 0: No geolocation or hunt mode implemented.
-
- AR/VR, Camera, and Mic Integration (8 points)
 - 8: AR overlays Pokémon on camera (via ViroReact); VR-lite 360-view using gyroscopic controls; voice search and photo capture fully functional.
 - 6: AR/VR partially implemented (e.g., basic overlay or gyro view); camera/mic work with minor bugs.
 - 4: Limited AR/VR (e.g., static images); camera or mic partially functional.
 - 2: Minimal AR/VR or camera/mic attempted but broken.
 - 0: No AR/VR, camera, or mic features.
 - Sharing, Multimedia, and Extras (4 points)
 - 4: Social sharing (via react-native-share) and community feed work; GIFs/images load efficiently; optional gamification (badges/challenges) included.
 - 3: Sharing and feed functional with minor bugs; images load; no gamification.
 - 2: Basic sharing or image loading with issues; no feed or gamification.
 - 1: Minimal sharing or media attempted but broken.
 - 0: No sharing, multimedia, or extras.

2. Technical Quality (30 points)

Assesses code quality, performance, and cross-platform compatibility.

- Code Structure and Maintainability (10 points)
 - 10: Well-organized React Native code; modular components; clear file structure; proper use of Redux/Context; .env for secrets.
 - 8: Organized code with minor inconsistencies; state management used; secrets mostly secured.
 - 6: Code functional but messy; state management partial; secrets exposed.
 - 4: Poorly structured code; minimal modularity; no state management.
 - 0: Unreadable or non-functional code.
- Cross-Platform Compatibility (10 points)

- 10: App runs smoothly on both iOS and Android; permissions (location, camera) handled correctly via react-native-permissions.
 - 8: Runs on both platforms with minor UI/rendering issues; permissions mostly handled.
 - 6: Runs on one platform well, other has issues; permissions partially handled.
 - 4: Runs on one platform only or significant issues on both.
 - 0: Does not run on either platform.
- Performance and Optimization (10 points)
 - 10: App performs well on mid-range devices; API calls optimized (e.g., batching); images cached; no crashes.
 - 8: Good performance with minor slowdowns; some optimization (e.g., caching attempted).
 - 6: Noticeable lag or crashes; minimal optimization.
 - 4: Poor performance; frequent crashes; no optimization.
 - 0: App unusable due to performance issues.

3. Usability and Design (15 points)

Evaluates the user interface and experience.

- UI/UX Design (10 points)
 - 10: Clean, intuitive UI; Pokémon-themed design; consistent navigation (React Navigation); responsive across devices.
 - 8: Functional UI with minor inconsistencies; navigation clear; theme applied.
 - 6: Basic UI; navigation confusing; theme partially applied.
 - 4: Poor UI; navigation broken; minimal theme.
 - 0: No cohesive UI or unusable design.
- User Experience and Accessibility (5 points)
 - 5: App is intuitive; clear feedback (e.g., loading states); basic accessibility (e.g., readable fonts, alt text for images).
 - 4: Intuitive with minor UX issues; limited accessibility.
 - 3: Usable but confusing; no accessibility considerations.
 - 2: Poor UX; difficult to navigate.
 - 0: Unusable or no UX design.

4. Documentation and Presentation (10 points)

Assesses the submitted materials and teamwork.

- Documentation and Code Repository (5 points)
 - 5: GitHub repo well-organized; clear README; code comments explain key logic; report details challenges and solutions.
 - 4: Repo organized; README basic; report covers challenges partially.
 - 3: Repo exists but messy; minimal README or report.
 - 2: Repo incomplete; no report or README.
 - 0: No repo or documentation.
- Demo Video and Presentation (5 points)
 - 5: Demo video (2-5 min) clearly shows all features; explains implementation; all group members contribute.
 - 4: Video shows most features; minor clarity issues; group contribution evident.
 - 3: Video shows some features; unclear or rushed; uneven contribution.
 - 2: Video incomplete; minimal explanation; poor group effort.
 - 0: No video or no group effort.

5. Teamwork and Process (5 points)

Evaluates collaboration and development process.

- Team Collaboration and Agile Process (5 points)
 - 5: Evidence of equal contribution (e.g., Git commits); agile sprints with clear milestones; regular progress updates.
 - 4: Good collaboration; minor uneven contributions; some agile practices.
 - 3: Uneven contributions; minimal agile process.
 - 2: Poor collaboration; no clear process.
 - 0: No evidence of teamwork or process.

Bonus Points (Up to 5 points)

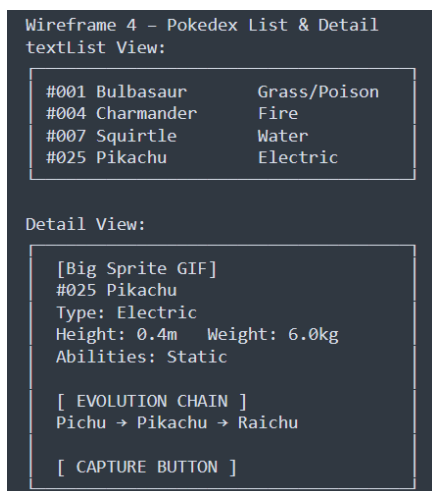
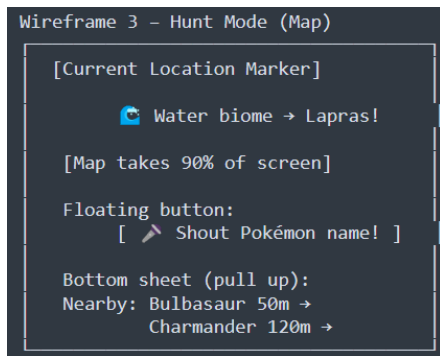
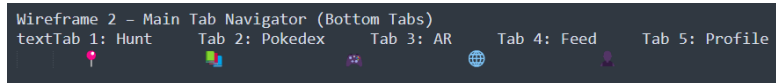
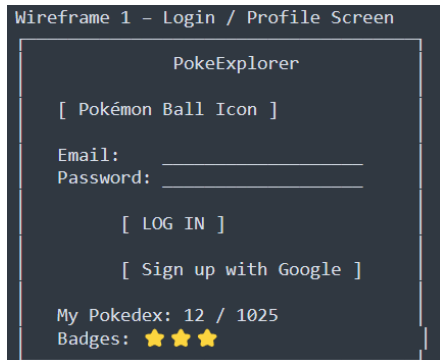
- Up to 5 points for exceptional creativity or features beyond requirements (e.g., multiplayer via WebSockets, advanced AR effects, or innovative gamification).

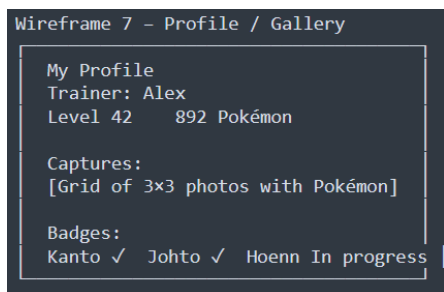
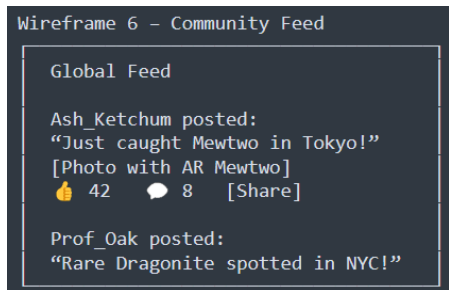
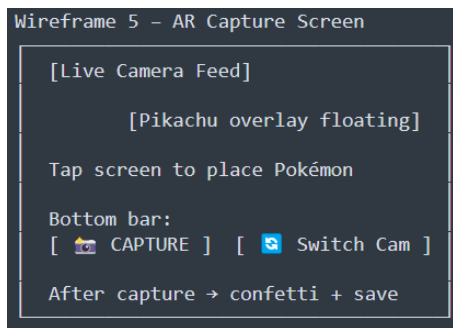
- Points awarded only if core requirements are met.

Total: 100 points + 5 bonus points

APPENDIX

Samples, you may use as reference or you can design your UI/UX based on below.





Figma Link

Top Recommendation: Mobile Pokédex Prototype (Exact Match for Your Specs)
<https://www.figma.com/community/file/979132880663340794/pokedex>

- Includes: Login, Pokedex list/detail, search, evolution chains, AR camera overlay mockup, map hunt screen, profile/gallery, sharing feed.
- Has style guides, components, wireframes, and full flows. Perfect for bare React Native implementation.
- 10/10 fit—students can directly code from this.

🔗 Another Great One: Pokedex with Map & Capture Flows

<https://www.figma.com/community/file/1322325075311624960/pokedex>

- Strong on geolocation map, AR capture, and community sharing screens.

🔗 Bonus: Kanto Pokédex Gen I (Simple & Clean for Quick Start)

<https://www.figma.com/community/file/1170325929062335042/pokemon-kanto-pokedex-gen-i>

- Great for the core list/detail + profile badges.

More Options (Full List): Search Figma Community for "Pokedex" →

https://www.figma.com/community/search?resource_type=files&sort_by=popular&query=pokedex&editor_type=all (Over 50 free templates—filter by "Mobile" and "Wireframe".)